

DESIGN_DECISION

BOB-174 — Design decision: how a CORRUPT hook store behaves, and why

Revision: 1 **Last modified:** 2026-08-23T09:55:00Z **Item:** BOB-174 (Bug, High) — a corrupt hooks file reads as zero hooks, and the next create destroys the rest **Scope of this document:** the decisions the item required to be *recorded rather than inferred*, and the reasoning and measurements behind each.

0. The root, stated once

`_load_hooks` had one return value — `[]` — for two states with opposite meanings:

State	What it means	Correct answer
hooks.json MISSING	No hooks configured. The fresh-install state.	<code>[]</code> — genuinely empty
hooks.json EXISTS but unreadable	The configured hooks are UNKNOWN to the server. They may still be firing.	<i>not</i> <code>[]</code> — the server cannot answer

Collapsing them is the whole defect. Everything below follows from separating them.

This is a §11.4.201(6) **FALSE-NULL**: a blind instrument and a clean artifact returned the identical quiet zero. The fix is not “handle the exception better” — it is to make the two states **distinguishable at the API surface**, which is why a new exception type (`HookStoreCorruptError`) was introduced rather than a sentinel value or a `None` return.

1. DECISION — GET returns 500, not 200 with a degraded marker

Chosen: GET /api/v1/hooks answers a corrupt store with **HTTP 500** and a detail naming the file and the consequence. The error body carries **no hooks key at all**.

Rejected: 200 {"hooks": [], "count": 0, "degraded": "..."}.

The measurement that decided it

The item flagged that this endpoint is consumed by the frontend, so a bare 200→5xx flip has UI consequences. It was checked rather than assumed.

frontend/src/app/components/dashboard/dashboard.component.ts:794:

```
loadHooks(): void {  
  this.api.getHooks().subscribe(h => {  
    this.hooks.set(h.hooks || []);  
  });  
}
```

subscribe() is called with a **next callback only** — no error callback. getHooks() pipes only timeout(...); there is no catchError, no HTTP interceptor and no custom ErrorHandler anywhere in the non-spec frontend sources (verified by grep over frontend/src/app, 2026-08-23). The template renders <tr *ngIf="hooks().length === 0">No hooks registered</tr>.

So, with the frontend **as it stands today**:

Option	What the hooks signal does	What the operator sees	Any signal at all?
500 (chosen)	never updated — stream errors	"No hooks registered"	yes — RxJS rethrows to Angular's default ErrorHandler → console.error, plus a red row in the Network tab
200 + marker	h.hooks \ \ [] → []	"No hooks registered"	no — a clean 200, nothing anywhere

Both render the same wrong sentence today. That is the honest finding, and it is why the argument had to be decided on something other than the current UI. Three things decided it:

1. **A marker only helps consumers that opt in to reading it.** Every consumer that exists today — the dashboard, curl, any operator script — checks the status or nothing. A new field they do not read is silence, which is the §11.4.201(6) shape the item exists to remove. A 500 is understood by every HTTP consumer that already exists, with no coordinated change.
2. **count: 0 on a 200 is a false statement on the wire.** It is not merely uninformative; it is the original lie, re-emitted with a footnote. An error response with no hooks key cannot be misread by a consumer that ignores the status, because there is nothing there to misread — which is why the guard asserts "hooks" not in body.
3. **The 500 leaves a trace even untouched.** A console error and a red network row are not a good UI, but they are a *distinguishable* state, and they are what turns “the hooks tab looks empty” from a dead end into a five-second diagnosis.

Status code: 500, not 503

503 means “temporarily unavailable, retry may help”. A corrupt file does not repair itself and a retry will fail identically; 503 would invite exactly the wrong response. 500 also matches what BOB-173 already established for the write-failure path in this same module, so the two persistence faults present consistently.

HONEST BOUNDARY — what this decision does not fix

The dashboard still shows “No hooks registered” on a corrupt store. This change makes the *API* honest; it does not make the *UI* honest. Making the UI render a distinct degraded state requires an error callback in `loadHooks()` and a template branch — both in `frontend/`, which is outside this item’s file scope and live under a sibling stream. That is a **known, unfixed gap**, recorded here so it is a tracked decision rather than an oversight, and it is the natural follow-up item. The API contract chosen above is deliberately the one that makes such a frontend fix a pure addition (read the status) rather than a renegotiation (agree on a new field).

2. DECISION — create and delete refuse; they never clobber

POST and DELETE both answer a corrupt store with **500** and change nothing on disk.

This is §11.4.252 applied literally: the path combines **mutation of a shared resource** (rewriting the store) with an **external side effect** (a hook is an outbound call the system will or will not later make), so when an input to its correctness is unverifiable it **fails closed**.

The alternative — “read what we can, write what we have” — is precisely the defect. Pre-fix, create received [] from a misread of a three-hook file, appended one hook, and wrote a one-element list over the top. Measured on the fixed-but-for-this tree:

```
'prod-hook-0' is no longer in the hook store.  
assert 'prod-hook-0' in ' [{"hook_id": "f71c5ebb-...", "name":  
"bob174-new-hook", ...}] '
```

Note what is destroyed. A truncated file still contains the operator’s hook definitions in plain text — it is **hand-repairable**. After the clobber it is not. The create turned a recoverable state into an unrecoverable one, and reported 200.

test_recoverable_operator_data_is_still_recoverable is the guard that pins exactly that property.

DELETE gets the same treatment for a second reason: its 404 is a *claim about the file’s contents* (“Hook not found”). Making that claim about a file that was never successfully read is a false report, not a null result — pre-fix, deleting a hook that **was** in the store answered 404.

3. DECISION — dispatch_event logs and dispatches nothing; it does not raise

This is the one place a log-only arm was accepted, and it is the decision most in tension with the rest of the item, so the reasoning is recorded in full.

dispatch_event is awaited **inline** by the search and download handlers (api/routes.py, 14 call sites). Raising HookStoreCorruptError out of it would turn an unreadable hooks file into a **500 on SEARCH and on DOWNLOAD** — a §11.4.201(1) false-positive refusal that breaks the product’s primary capabilities over an unrelated subsystem’s state. That is strictly worse than the gap it would close.

So the chosen behaviour is: **dispatch no hooks** (fail closed on the side effect — the set of hooks to run is unknown, so none run), **log an ERROR** naming the file, and **return**.

HONEST BOUNDARY (§11.4.6): a log-only arm is the exact shape this item condemns everywhere else. It is accepted *here and only here* because `dispatch_event` is fire-and-forget and has **no response channel of its own to be honest on**. `GET /api/v1/hooks` is the surface where the operator learns the store is broken, and it now says so loudly. If a future change gives the dispatch path a status surface (a health field, an SSE event, a hook-execution log entry), that surface should carry this state and this arm should stop being log-only.

4. DECISION — “corrupt” includes wrong shape, bounded at list-of-objects

`HookStoreCorruptError` is raised for:

Input	Why
unreadable file (<code>OSError</code>)	exists, contents unobtainable
truncated / malformed JSON (<code>ValueError</code>)	the crash-mid-write case A5 produces
valid JSON that is not a list (<code>{"hooks": [...], "x", 42}</code>)	parses cleanly; a fix hardening only <code>json.load</code> would still collapse it to <code>[]</code>
a list containing non-objects (<code>["a", "b"]</code>)	every accessor in this module is <code>h["hook_id"] / h.get("event")</code>

And, found by self-review rather than by the reported chain: a **directory** (or `fifo/socket/device`) at the store path. `os.path.isfile` is `False` for those, so the original guard — and any fix that kept `isfile` — reported them as `MISSING`, i.e. “no hooks configured”. That is the same conflation in a rarer flavour, so the guard is now `os.path.exists`, letting such a path fall through to the open where the existing `OSError` handler covers it. A broken symlink deliberately stays `MISSING` (`exists` follows links) — nothing readable is there, and that reading is correct.

Deliberately NOT validated: individual keys within each hook object. A hook missing `hook_id` would still fail later in `delete_hook`. That was left alone on purpose — per-key validation risks refusing hand-edited-but-workable files and files carrying fields added by a future version, which is a §11.4.201(1) false-positive refusal of a *healthy* store. The line drawn is: **“could the contents be obtained in the shape this module contracts for”** — not “is

every hook well-formed”. Stricter validation is a defensible future change, but it is a different decision with a different risk profile and should be made on its own evidence.

HookStoreCorruptError is kept **distinct** from BOB-173’s HookPersistenceError rather than merged: one means a READ failed, the other a WRITE. They arrive at different points and an operator repairs them differently. Collapsing them would recreate, one level up, the same “two states, one signal” defect this item exists to remove.

5. DECISION — the atomic write is theme_state’s pattern, reused

_save_hooks now writes via tempfile.mkstemp → json.dump → flush → fsync → os.replace, with the temp file unlinked on any failure.

This is **not invented here**. It is the pattern already established in this codebase by download-proxy/src/api/theme_state.py::_write_atomic (§11.4.28 — reuse, do not re-derive). The temp file is created **in the destination directory**, not /tmp, because os.replace is only atomic within a filesystem; a cross-device rename would silently reintroduce the window being closed.

It was **not** extracted into a shared helper, which is the obvious right end-state, because that means editing theme_state.py — outside this item’s file scope. Recorded as a follow-up.

Why fix the write at all when A1 now catches the corruption it produces? Because A5 is the link that **manufactures** the corrupt file. Guarding only the read leaves every crash mid-write destroying the store and leaves the operator with a repair job; the guard downstream turns silent data loss into a visible 500, which is better but is not the same as not losing the data.

DECISION — the permission-semantics change this carries, accepted knowingly

An atomic write changes *which permission governs*. Measured, not assumed:

```
append to a 0400 file : REFUSED (Permission denied)
os.replace over 0400  : SUCCEEDS
mkstemp in a 0500 dir : REFUSED (Permission denied)  <- the
atomic write fails here
```

So a **hooks.json chmod'ed 0400 no longer refuses a write**. `os.replace` relinks a directory entry and never consults the target file's mode; what governs now is write permission on the **directory**.

This was surfaced by three of BOB-173's DELETE tests going red — that file sealed the *file* and probed with `open(target, "a")`, a needle that matched the old in-place rewrite and no longer matches the code (the inverse of the §11.4.199 false reproduction its own header warns about). It was investigated before anything was touched, and the change is **accepted deliberately**, because:

1. File mode on the application's own state store is not a documented protection mechanism anywhere in this project — nothing sets it, nothing relies on it.
2. `theme_state.py` already behaves exactly this way; the alternative would leave two state stores in the same service with different durability guarantees.
3. The realistic deployment failure — the EACCES-on-/config case BOB-135 actually hit — is a **directory** permission problem, and it is still caught. That is why BOB-173's *create* tests never failed: only the file-sealed delete ones did.
4. The thing traded away (an undocumented `chmod`-based lock) is worth far less than the thing gained (the store can never be truncated by an interrupted write).

BOB-173's seal was reconciled to seal the directory using the needle that matches what the code now does (§11.4.120 — assert the NEW mechanism), and the premise above is pinned by a test rather than left as a docstring claim, so reverting the atomic write cannot silently invalidate the seal choice.

6. What is NOT claimed

- **Not claimed:** that the write currently fails in the operator's deployment, or how often. The chain is reproduced under controlled conditions; its real-world frequency is unmeasured.
 - **Not claimed:** that the dashboard now shows anything useful on a corrupt store. It does not — see §1.
 - **Not claimed:** that a corrupt store is now impossible. The atomic write closes the crash-mid-write vector this code owns; external corruption (a bad hand-edit, filesystem damage, a restore from a truncated backup) remains possible, which is exactly why the read guard exists as well.
 - **Not verified:** behaviour under a real ENOSPC or a real SIGKILL mid-write. The reproduction injects a partial write followed by `OSError(ENOSPC)`, which models the failure faithfully at the point that matters (the file is truncated at open before serialisation completes) but is not the syscall-level event itself.
-

7. A3 — the event-vocabulary pin: pinned, not deduplicated

VALID_EVENTS (api/hooks.py) and HookEventType (merge_service/hooks.py) are two sources of truth for one closed set. **Verified identical today** (both hold the same 8 values).

The drift consequence is worth stating because it is silent: create_hook validates against VALID_EVENTS, while dispatch_event resolves HookEventType(event_type) first and **returns early** on ValueError. An event present in one and not the other therefore registers with **HTTP 200** and then **never fires**, logging a warning nobody reads.

Decision: pinned by assertion, not unified.

TestEventVocabularyCannotDrift asserts set equality and that every accepted event is dispatchable. Unifying the two — deriving VALID_EVENTS from the enum — is the better end-state but requires editing merge_service/hooks.py, which is outside this item's scope and live under a sibling stream. The pin costs one assertion and makes the drift impossible to land silently, which is the property that was missing.

Guard

tests/unit/api_layer/test_bob174_corrupt_hook_store.py — 28 tests. Evidence, including the observed RED, the GREEN, and five paired \$1.1 mutations, is in EVIDENCE.md beside this file.